

KatallIA: An LLM-Powered Refactoring Agent Guided by Design Patterns, Structural Metrics, and Code Smells

1st Diego Gatica Pizarro
Escuela de Ingeniería Informática
Universidad de Valparaíso
Valparaíso, Chile
diego.gatica@uv.cl

2nd Rene Noel
Escuela de Ingeniería Informática
Universidad de Valparaíso
Valparaíso, Chile
rene.noel@uv.cl

Abstract—Legacy systems accumulate structural erosion (e.g., complexity, duplication, and coupling), increasing change effort and regression risk. Existing automated approaches address this problem only partially: static analysis provides diagnosis without action, search-based methods explore transformations without explicit design intent, and LLM-based approaches often lack grounding, control, and reproducibility. This paper presents *KatallIA*, a maintainability-aware refactoring recommender that integrates metric-based diagnosis, smell detection, hotspot ranking, and LLM-guided refactoring synthesis into a unified, evidence-driven pipeline. *KatallIA* enforces explicit guardrails (SAFE-first ordering, bounded scope, and build preservation) to ensure conservative, incremental, behavior-preserving transformations while maintaining traceability and reproducibility. We evaluate *KatallIA* through an industrial case study on a Python legacy service. Applied to the main hotspot, Maintainability Index increased from 31.92 to 59.11, duplication was substantially reduced and complexity decreased at the hotspot level, and expert review judged recommendations relevant and actionable, supporting the feasibility of low-risk, evidence-driven refactoring in practice.

Index Terms—component, formatting, style, styling, insert

I. INTRODUCTION

Legacy software systems continue to support critical operations in domains such as banking, telecommunications, and healthcare, both in Latin America and globally [1]. Although often built using outdated technologies and architectural styles, these systems remain in production due to high replacement costs and strong business dependence. Over time, however, accumulated structural erosion—manifested as rigid designs and tight coupling—makes maintenance and safe evolution increasingly difficult.

Maintainability has been studied from multiple complementary perspectives, including quality models, structural metrics, and modernization practices. ISO/IEC 25010 defines maintainability as the degree to which a system can be modified effectively and efficiently, encompassing modularity, analysability, and modifiability [2]. In practice, these properties are often approximated through static structural indicators such as Cyclomatic Complexity (CC) and the Maintainability Index (MI), which serve as proxies for change effort and

understandability [3]. Complementarily, refactoring techniques and design patterns have long been advocated as mechanisms to reduce coupling, improve modularity, and enable more maintainable designs [4].

Refactoring operationalizes maintainability improvement through behavior-preserving transformations that incrementally reshape internal structure while keeping externally observable behavior stable. Recent Large Language Models (LLMs) have shown promising capabilities for code understanding and transformation, opening new opportunities to assist legacy modernization [5]. However, existing automated approaches address this challenge only partially: search-based techniques can explore large refactoring spaces but rely on predefined fitness functions without semantic understanding of design intent [6]; static analysis tools detect smells and metric violations systematically yet stop at diagnosis without generating actionable transformation steps [7]; and LLM-based pipelines, while flexible, typically operate under loosely constrained prompting strategies and lack explicit mechanisms to enforce alignment with structural quality objectives. Consequently, generated transformations may exhibit semantic inconsistencies, incomplete refactorings, or weak grounding in maintainability-related evidence such as metrics or code smells [5], [8]. Critically, generative models operate as probabilistic and non-deterministic systems, often lacking transparency and standardized evaluation practices [9]. When applied without explicit grounding in structural metrics, design patterns, or quality objectives, their outputs become difficult to reproduce, validate, and trace—introducing risks analogous to making architectural decisions without explicit engineering principles, particularly in legacy or safety-critical systems. As a result, practitioners still lack integrated and reproducible artifacts that combine diagnosis with constrained, pattern-guided or refactoring-guided transformations under explicit risk guardrails [7].

To address this gap, we propose *KatallIA*, a maintainability-aware refactoring recommender organized as a four-stage pipeline: (i) metric-based diagnosis, (ii) code and design smell detection, (iii) hotspot ranking, and (iv) refactor-

ing synthesis. In KatalIA, improvements are operationalized through behavior-preserving refactoring techniques (e.g., Extract Method, Move Method, Introduce Parameter Object) and through design-pattern interventions (e.g., Strategy, Dependency Injection) represented explicitly as refactoring sequences. Refactoring candidates are generated by an LLM that is grounded on a structured knowledge base linking metrics, smells, refactoring operations, and patterns, and are then prioritized and filtered according to evidence from diagnosis and explicit change-scope guardrails, aiming to keep modifications localized and conservative.

The proposed approach is validated through an industrial case study conducted on a Python project from a Chilean financial company, following a reproducible protocol based on the case study guidelines [10]. The evaluation assesses the end-to-end performance of the KatalIA pipeline using pre/post comparisons on maintainability indicators under explicit risk guardrails (bounded churn, build process must pass, and behavior preservation). In addition to tool-based measurements—including structural metrics, smell detectors, and Git/CI mining—the results are triangulated with an expert-based assessment: a software engineer with ten years of professional experience reviews and evaluates the relevance, correctness, and practicality of the recommended and applied refactorings.

This paper contributes: (i) the design of KatalIA as a hybrid, maintainability-aware refactoring recommender, (ii) an evidence-driven workflow that integrates metrics and code/design smells with explicit decision protocols and regression-risk guardrails, and (iii) a reproducible industrial evaluation protocol combining quantitative indicators and expert judgment.

The remainder of this paper is organized as follows. Section II reviews prior work on software maintainability, refactoring techniques, design patterns, and the use of large language models in software modernization. Section III presents the proposed KatalIA approach, including its architecture and refactoring pipeline. Section IV describes the empirical study design, the industrial context, the evaluation protocol adopted to assess the artifact, and replication package. Section V reports the results of the case study, followed by a discussion of the findings. Finally, Section VI concludes the paper and outlines directions for future work.

II. RELATED WORK

Several strands of work are relevant to this study: (i) legacy refactoring and modernization practices, (ii) the empirical relation between structural metrics and code smells, (iii) the interaction between design patterns and smells, and (iv) the use of large language models to assist refactoring and modernization. This section briefly summarizes these strands and positions KatalIA with respect to them.

A common approach to legacy refactoring is the systematic application of design principles such as SOLID [11]. Industrial evidence suggests that introducing SOLID principles can enable effective code restructuring, reducing coupling

and increasing cohesion, with positive effects on testability [12]. At the behavioural level, modernization efforts have also employed model learning and equivalence checking to ensure that observable behaviour remains unchanged after refactoring, highlighting the importance of behavioural safeguards [13]. From a risk management perspective, code churn—the volume of modifications over time—has been established as a predictor of defects and maintenance burden [14], motivating the use of bounded churn constraints to limit the scope of automated refactorings and control regression risk [15].

From a more algorithmic angle, search-based software engineering has explored evolutionary techniques to generate sequences of refactorings that improve cohesion and reduce coupling, showing that automated search can outperform manual efforts on large systems [6].

A complementary line of work empirically connects structural metrics, code smells, and defect proneness. Studies show that thresholds on CK-style metrics (e.g., WMC, CBO, RFC) can be calibrated to support smell detection and fault prediction in industrial-strength systems [16]. Other work further analyses which smells are effectively detectable via metric-based approaches, using object-oriented metrics and statistical calibration [17]. At a system level, smell density has been treated as an indicator of maintainability, with evidence that certain smells remain associated with change-proneness even after controlling for system size and other confounders [18]. Together, these studies motivate the combined use of metrics and smell-based diagnosis as a front-end for refactoring support, rather than relying on ad-hoc intuition.

The relationship between design patterns and code smells has also been explored empirically. Prior analyses report that some GoF patterns correlate with lower smell density, whereas others tend to co-exist with recurring design defects [19]. This suggests that patterns are not automatically “good” or “bad”; their impact depends on how and where they are applied. Accordingly, patterns and smells should be considered jointly—as both risk indicators and refactoring targets—rather than in isolation. This assumption underpins KatalIA’s pattern-guided recommendations layered on top of metric- and smell-based diagnosis.

Recent advances in large language models have opened new opportunities to support software modernization. A systematic literature review of LLM-driven refactoring reports heterogeneous tool-supported pipelines, a prevalence of Java-centric datasets, sensitivity to prompting strategies, and a lack of standardized definitions of refactoring *accuracy*; it also highlights recurring limitations such as erroneous code generation and difficulties with complex refactorings, motivating the use of constraints, tooling, and evidence-backed workflows [5]. Complementary empirical evidence shows that LLM effectiveness can increase substantially when refactoring tasks are framed with more explicit guidance (e.g., refactoring subcategories) and evaluated against real refactoring instances [8]. More advanced architectures leverage specialized LLM agents, suggesting that explicit task decomposition can improve the quality of software engineering tasks compared to single-

model baselines [20]. In parallel, empirical studies have shown that LLMs can effectively support automated refactoring tasks by improving code structure and readability [8]. Together, these findings suggest that combining task decomposition with code transformation capabilities may enable more contextual and quality-driven refactoring pipelines.

Despite the progress across these strands, current approaches exhibit several systemic limitations when considered jointly. First, metric- and smell-based techniques provide robust diagnostic capabilities but typically stop short of generating actionable, context-aware refactoring steps. Second, search-based approaches can explore large transformation spaces but rely on predefined fitness functions, often lacking explicit alignment with maintainability objectives or design intent. Third, LLM-based approaches, while flexible, frequently operate without strong grounding in structural evidence, leading to non-deterministic outputs, inconsistent transformations, and limited reproducibility. Finally, agent-oriented approaches emphasize task decomposition but remain primarily focused on analysis or localization, without systematically closing the loop with behavior-preserving transformations. As a result, existing solutions lack integrated, reproducible mechanisms that connect diagnosis, prioritization, and constrained refactoring synthesis under explicit risk control.

Table I synthesizes these differences across key dimensions derived from the literature. Diagnosis refers to the ability to identify structural issues, while Action indicates whether the approach produces concrete code transformations. Grounding captures whether decisions are explicitly supported by structural evidence, and Risk Control reflects the presence of mechanisms to constrain change scope or prevent regressions. The symbols denote the level of support: ✓ indicates full support, × indicates absence, and ~ denotes partial or indirect support.

While not always explicitly defined as standalone dimensions in prior work, Scope and Determinism are derived from the operational characteristics of each approach. Scope characterizes the extent of modifications: Local refers to changes confined to a limited code region, Global denotes system-wide transformations typically explored through search, Flexible reflects unconstrained or context-dependent changes typical of LLM-based approaches, Task-level refers to decomposition at the level of analysis rather than direct code transformation, and Bounded indicates explicitly constrained modifications guided by selection policies and risk-control guardrails. Determinism captures reproducibility: High corresponds to rule-based or deterministic analyses, Low to stochastic generation, and intermediate values indicate partial control through constraints or guided processes.

Overall, prior work shows that principle-based and search-based refactoring can improve structural quality, and that metrics and code/design smells are empirically linked to maintainability and change proneness. However, as summarized in Table I, these approaches remain fragmented across diagnosis, transformation, and control dimensions. KatalIA addresses this disconnect by turning diagnosis into action: metric- and smell-

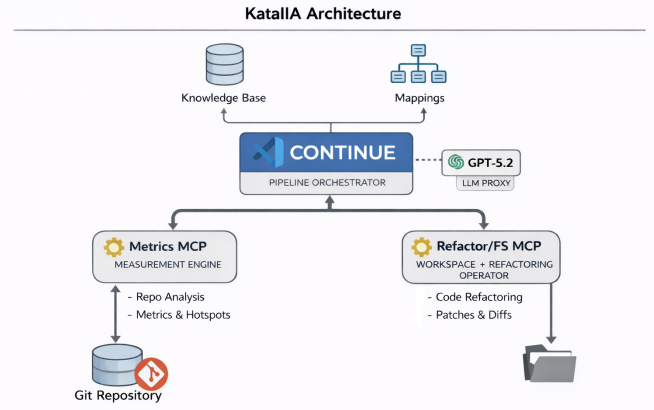


Fig. 1. KatalIA architecture. The developer interacts through Visual Studio Code using the Continue extension. Continue mediates communication between the LLM (GPT-5.2) and a set of specialized MCP servers connected to the software repository.

driven hotspot selection constrains and prioritizes refactoring and pattern-level interventions synthesized by an LLM under explicit risk guardrails, producing conservative, evidence-backed recommendations rather than ad-hoc transformations.

III. PROPOSED SOLUTION

This work proposes *KatalIA*, a hybrid, maintainability-aware refactoring approach that integrates evidence-driven diagnosis with LLM-guided refactoring synthesis. KatalIA addresses the fragmentation identified in prior work by unifying diagnosis, prioritization, refactoring action, grounding, and risk control within a single pipeline. Unlike existing approaches that typically focus on isolated dimensions, KatalIA connects structural evidence (metrics and smells) with constrained, behavior-preserving transformations under bounded scope and reproducibility-oriented guardrails, enabling conservative and localized improvements while controlling regression risk.

A. System Architecture

Figure 1 presents the high-level architecture of KatalIA. The system is designed as an IDE-embedded, agent-oriented refactoring assistant that tightly integrates large language models (LLMs), static analysis services, and the developer's working context.

At the interaction layer, KatalIA is embedded within *Visual Studio Code* through *Continue*, an open-source AI coding assistant that provides native access to the IDE state. Continue exposes fine-grained contextual information to the LLM, including open files, cursor position, selected code fragments, repository structure, and configuration metadata. This enables refactoring decisions to be grounded in the actual development context rather than isolated code excerpts.

Continue also acts as the orchestration layer for external analytical capabilities through the *Model Context Protocol* (MCP). MCP is a standardized protocol for exposing tools,

Approach	Diagnosis	Action	Grounding	Risk Control	Scope	Determinism
Metric/Smell-based [16]–[18]	✓	×	✓	✓	Local	High
Search-based [6]	✓	✓	~	~	Global	Medium
LLM-based [5], [8]	~	✓	×	×	Flexible	Low
Agent-based LLM [20]	✓	~	~	×	Task-level	Low
KatalIA (proposed)	✓	✓	✓	✓	Bounded	Medium–High

TABLE I
COMPARISON OF EXISTING APPROACHES AND KATALIA ACROSS KEY DIMENSIONS.

data sources, and services to LLM-based agents in a structured and composable manner. In KatalIA, MCP servers encapsulate program analysis and repository mining functionality, such as metric computation, smell detection, clone analysis, dependency graph extraction, and change-history analysis—a key contribution that grounds LLM reasoning on rigorous static analysis rather than pattern recognition alone. These services operate directly on the source repository and return machine-readable artifacts (e.g., JSON), which are subsequently interpreted by the LLM.

The reasoning layer of the system relies on the GPT-5.2 large language model. The choice of GPT-5.2 is motivated by its strong capabilities in multi-step reasoning, tool invocation, and synthesis of heterogeneous evidence—properties that are critical for non-trivial refactoring and design-level decision making. Prior empirical studies have shown that advanced LLMs significantly outperform earlier generations in tasks requiring contextual understanding, structured reasoning, and interaction with external tools, particularly in software engineering settings [21]. Moreover, recent research in software engineering has demonstrated the effectiveness of LLMs when combined with static analysis and program representations, rather than relying on purely generative behavior [22].

In KatalIA, the LLM does not perform code analysis directly. Instead, it acts as a reasoning agent that consumes evidence produced by MCP-based analyzers and synthesizes refactoring recommendations, trade-off analyses, and architectural insights. This separation of concerns ensures that refactoring decisions are both explainable and traceable to concrete code-level measurements.

Overall, the architecture combines IDE-level situational awareness (Visual Studio Code¹ and Continue²), extensible and verifiable analysis (MCP services), and state-of-the-art reasoning capabilities (GPT-5.2³). This design allows KatalIA to support incremental, low-risk refactorings while remaining scalable to system-level architectural assessments.

B. KatalIA Pipeline

KatalIA is organized as a four-stage pipeline that incrementally transforms structural evidence into refactoring actions.

The pipeline is intentionally modular to allow independent evolution of analysis, decision, and transformation stages.

The first stage performs *metric-based diagnosis*. KatalIA computes a catalog of **126 structural metrics**⁴ [23] spanning multiple *scopes* (*function/method, class, file, component, repo*) and organized by *category* (e.g., Entities, Size/Complexity, Encapsulation, Architecture, Clones, Churn, Git History, Advanced Design). Rather than acting as isolated indicators, metrics are treated as complementary signals used to characterize maintainability-related properties in context.

The second stage applies *code and design smell detection*, lifting raw metric signals to higher-level structural symptoms associated with maintainability and evolvability problems. Smells function as interpretable abstractions over metric profiles, enabling diagnostics beyond individual thresholds. The full mapping between smells and their associated metric signals, including literature-based justifications, is documented in the same Zenodo repository [23].

The third stage performs hotspot-based candidate selection and prioritization. KatalIA computes a hotspot ranking from structural metrics, and selects candidates using signal-presence guards (e.g., $MI < 65$) and a bounded *top_k* policy (configurable; we used project-level defaults). We justify $MI < 65$ as a conservative *screening guard* based on a commonly adopted interpretation of the Maintainability Index (MI) in which values below 65 indicate low maintainability (65–85 medium; > 85 high) [24]. Because MI admits multiple formula variants and tool-dependent scalings, we do not treat this threshold as a universal maintainability verdict; instead, it is used purely as a pragmatic gate within a multi-signal selection bundle (combined with complementary size/complexity/duplication proxies and bounded *top_k*). This focuses the intervention scope on a small set of high-impact entities and avoids system-wide or indiscriminate refactoring. No LLM-based re-ranking is applied at this stage; change-scope constraints are enforced during selection by limiting the candidate set.

Finally, the fourth stage—refactoring synthesis—generates concrete improvement actions. At this stage, structural indicators and hotspot context are resolved against a curated knowledge base that encodes relationships between common issues, behavior-preserving atomic refactoring techniques (e.g., Extract Method, Move Method, Introduce Parameter Object),

¹<https://code.visualstudio.com/>

²<https://continue.dev/>

³<https://platform.openai.com/>

⁴<https://doi.org/10.5281/zenodo.18408185>

and GoF design patterns represented as structured sequences that realize higher-level design intent (e.g., Template Method, Strategy, Facade, Dependency Injection), as documented in the same Zenodo repository [23]. Candidate refactorings are proposed by an LLM operating as a constrained reasoning component over this knowledge base—rather than as an unconstrained code generator—and are classified into risk categories (SAFE, SAFE*, PATTERNS, HIGH RISK) before being filtered through change-scope constraints to keep modifications localized, incremental, and behavior-preserving.

C. Metric–Smell–Refactoring–Pattern Mapping

KatalIA relies on an explicit, machine-readable mapping encoded in YAML catalogs to transform structural evidence into refactoring actions. Raw analyses (metrics, AST, clones, history, and graphs) are interpreted as *signals* with declared scope and optional polarity/min-value guards. Smells are defined as interpretable predicates over these signals and emit an evidence bundle (entity, location, and observed signal values). Each smell is then linked to a constrained set of remediations: either an explicit sequence of behavior-preserving atomic refactorings (e.g., `EXTRACT_METHOD`, `MOVE_METHOD`) or a target GoF pattern (tradeoffs-only catalog) to be realized through atomic refactorings following refactoring-to-pattern guidance. This structure ensures traceability: *entity* $\rightarrow$ *signals* $\rightarrow$ *smell* $\rightarrow$ *remediation* (sequence or pattern).

D. Prompt Design and Usage

KatalIA uses three predefined prompts that correspond to the main stages of the workflow. A diagnostic prompt guides the interpretation of the metric and smell artifacts, a synthesis prompt generates candidate refactorings grounded in the mapped refactoring techniques and GoF patterns, and a post-synthesis prompt re-evaluates the same metric set after the changes to verify whether the expected structural effects are observed.

The prompts were iteratively refined to improve the LLM’s precision and reduce unsupported reasoning. In particular, they constrain the model to operate only over the available evidence (computed metrics and detected smells) and the curated mappings between smells, refactoring techniques, and design patterns. The prompts are also explicitly oriented toward incremental and safe modifications, prioritizing conservative, localized, and behavior-preserving changes.

When the evidence admits more than one reasonable remediation, the prompts allow the system to propose alternatives: (i) applying refactoring techniques only, or (ii) applying a design pattern (represented as a structured sequence of refactorings). This enables practitioners to choose between a minimal mechanical improvement and a more explicit design-level improvement while keeping the intervention scope bounded.

All prompt definitions are available in the replication package on GitHub [25].

IV. EMPIRICAL STUDY DESIGN

This work adopts a *Design Science Research (DSR)* perspective and evaluates the KatalIA artifact through an *exploratory*

industrial case study. The objective of the study is not to establish definitive causal claims, but to obtain structured and reproducible evidence regarding the feasibility, usefulness, and potential impact of the proposed approach in a realistic legacy setting.

Following established guidelines for case study research in software engineering [10], [26], the study is designed as a *single-case, pre–post evaluation* without a control group. The unit of analysis is a legacy software system, and the focus is on understanding how KatalIA supports maintainability- and testability-oriented refactoring in an industrial context.

A. Case Study Context

The studied system is a web service developed in Python 3.9 following a modular monolithic architecture. It exposes approximately twenty REST endpoints implementing business logic that supports an online insurance sales process. The service is built using the Flask framework and relies on a relational PostgreSQL database for persistent storage. Data access is handled through SQLAlchemy, which is used as an Object–Relational Mapping (ORM) layer.

At the time of the study, the system did not include automated unit tests, nor did it provide static documentation such as architectural or component diagrams. Likewise, no dynamic documentation (e.g., sequence diagrams or runtime views) describing the system’s behavior was available. Consequently, system comprehension relied primarily on source-code inspection and static analysis.

The service is deployed on Amazon Web Services (AWS) using Elastic Kubernetes Service (EKS). Although the deployment infrastructure was not modified during the study, this information is reported to characterize the operational environment and to support the external validity of the case study.

B. Role of KatalIA and Practitioner

KatalIA was used by a professional software engineer with ten years of industry experience to perform the diagnosis and refactoring activities in the studied system. The involvement of an experienced practitioner aims to ensure that the applied refactorings are realistic and aligned with industrial software development practices rather than optimized for experimental conditions.

KatalIA assisted the engineer throughout the refactoring process by providing diagnostic information, prioritizing maintainability-critical hotspots, and suggesting refactoring techniques and, when applicable, candidate design patterns. Based on this analysis, KatalIA generated a prioritized refactoring plan.

The engineer reviewed the proposed plan and retained full control over all technical decisions. Due to time constraints, the refactoring effort focused exclusively on the highest-priority hotspot, which primarily involved local and design-level code smells. Although KatalIA suggested that the application of a design pattern was optional, the engineer decided to apply the suggested pattern as part of the refactoring. This

interaction model reflects a collaborative human–AI workflow, in which KatalIA provides structured guidance while professional judgment remains with the practitioner.

C. Case Study Procedure

The evaluation follows a three-step procedure:

- 1) **Baseline characterization (Pre).** Prior to refactoring, the system was analyzed to obtain a baseline snapshot of its structural properties. This included metrics related to complexity, coupling, cohesion, and size, as well as indicators related to testability and change effort, such as code churn, number of modified entities, and build execution behavior.
- 2) **Diagnosis and refactoring with KatalIA.** KatalIA was applied to identify and prioritize maintainability-critical hotspots using structural metrics and code/design smell detection. Based on this diagnosis, the artifact generated a refactoring plan guided by an explicit catalog of refactoring techniques and design-pattern-based refactoring sequences. The engineer reviewed the proposed plan and applied the refactorings for the highest-priority hotspot. Throughout this process, KatalIA continuously validated the system by compiling the software after relevant changes to ensure that no structural regressions were introduced.
- 3) **Post-intervention characterization (Post).** After completing the refactoring, the same set of metrics and indicators was re-collected. In addition, the engineer manually tested the system’s endpoints to verify that externally observable behavior was preserved.

The complete refactoring activity required approximately four hours, representing a realistic effort for an industrial refactoring task supported by an AI-based assistant.

D. Data Analysis

Data analysis is primarily descriptive and comparative. Pre-post differences are analyzed at both system and module levels using summary statistics and visualizations. Given the exploratory nature of the study, the emphasis is placed on transparency, traceability, and interpretation of observed trends and trade-offs rather than on statistical causal inference.

E. Research Questions

The case study is guided by the following research questions:

RQ1 (Diagnosis). To what extent do the structural hotspots identified by KatalIA (pre-refactoring) overlap with the files exhibiting the highest historical churn (pre-refactoring)?

RQ2 (Recommendations). To what extent does KatalIA transform its diagnosis (metrics and smells) into actionable and context-aware refactoring recommendations?

RQ3 (Outcomes). What changes in structural maintainability indicators and change size can be observed after applying KatalIA in an industrial legacy system?

Top- K	Churn	% Churn	Random (K/N)	Enrichment@ K
5	2555	37.8%	7.8%	4.84×
10	4014	59.4%	15.6%	3.80×
20	5407	80.0%	31.3%	2.56×
25	5870	86.9%	39.1%	2.22×

TABLE II
CHURN CONCENTRATION CAPTURED BY STRUCTURAL HOTSPOTS (PRE).
 $N = 64$ TRACKED PYTHON FILES; TOTAL CHURN = 6758.
ENRICHMENT@ K IS DEFINED AS CAPTURED@ $K/(K/N)$.

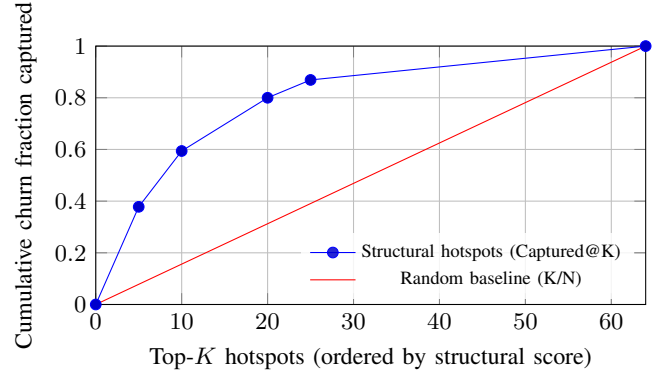


Fig. 2. Churn concentration curve (PRE): the Top- K structural hotspots capture substantially more churn than a uniform random selection (K/N), indicating that historical maintenance effort is concentrated in the files prioritized by KatalIA.

F. Replication Package

The replication package for this study includes the KatalIA tool, prompt definitions, and supporting documentation, and is publicly available on Github [25].

V. RESULTS AND DISCUSSION

a) RQ1 Results: The structural hotspot ranking shows a strong overlap with historical maintenance activity. Here, K denotes the inspection budget (i.e., the Top- K files ranked by structural score), and N denotes the total number of tracked Python files ($N = 64$). As summarized in Table II, the Top-10 structural hotspots capture $4014/6758 = 59.4\%$ of the total churn, corresponding to Enrichment@10 = $3.80\times$ relative to the uniform random baseline ($K/N = 15.6\%$). Enrichment@ K can be interpreted as a lift-over-random measure, obtained by normalizing Captured@ K by the expected fraction under random selection (K/N), analogous to the random-baseline comparison used in cumulative lift charts for effort-aware ranking in software quality prediction [27]. This concentration is already evident at smaller inspection budgets: the Top-5 captures 37.8% (Enrichment@5 = $4.84\times$), and the Top-20 captures 80.0% (Enrichment@20 = $2.56\times$).

Figure 2 reinforces this result visually: the cumulative churn captured by the Top- K hotspots (ordered by structural score) rises substantially faster than the random baseline line (K/N), indicating that maintenance effort is highly concentrated in the files prioritized by KatalIA.

At the file level, several top-ranked structural hotspots are also among the highest-churn files, including `app/adapters/repository/carro_repository.py` (churn_total=1103), `app/application/service/descarga_pdf_service.py` (1036), `app/application/service/carro_service.py` (914), and `app/adapters/handler/handler_carro.py` (538). Due to space constraints, the full Top-50 hotspot ranking and the complete churn tables (PRE) are available in the Zenodo replication package as supplementary material [23].

b) Discussion: Overall, these results suggest that KatalIA’s structural hotspot ranking aligns with where maintenance work has historically concentrated in the repository. In practice, a small inspection budget (e.g., Top-10 out of $N = 64$ files) already captures a large share of observed change activity, supporting its use as a prioritization aid for refactoring and quality improvement. The alignment is not expected to be perfect: churn is an empirical proxy that can reflect business-driven volatility or release timing, and structural risk may exist even when historical churn is low. Hence, discordant cases (high score/low churn or low score/high churn) should be treated as complementary signals, indicating latent maintainability risk or operationally critical modules, respectively.

c) RQ2 Results: KatalIA produced actionable, context-aware recommendations by translating diagnostic evidence (metrics and inferred smells) into a sequence of incremental steps under a SAFE-first strategy. Table III summarizes the executed steps for the refactored target `carro_repository.py`, including a controlled escalation from mechanical refactors to a single pattern application and a final modular extraction.

PR	Phase	Primary action
1	SAFE	Extraction + constants (duplication, magic numbers)
2	SAFE	Split-phase decomposition (<code>get_carro_by_id</code>)
3	SAFE*	Parameter object (long parameter list)
4	SAFE	Helper for repeated query <code>exec/fetch-one</code> flows
5	PATTERN	Template Method (stable variation)
6	HIGH-RISK	Extract reads/writes modules; facade delegation

TABLE III

RQ2 EXECUTION SUMMARY FOR `CARRO_REPOSITORY.PY`. SAFE* INDICATES MINIMAL CALL-CHAIN UPDATES DUE TO A SIGNATURE CHANGE.

Figure 3 complements this view by characterizing the execution scope (blast radius) of each step: most PRs were single-file, while scope expanded only when unavoidable (PR3 contract/signature change) and for an isolated modular extraction (PR6).

Practitioner feedback: An industrial practitioner reviewed the refactoring sequence and considered the detected smells and corresponding recommendations to be largely accurate. They highlighted that the SAFE-first ordering helped avoid

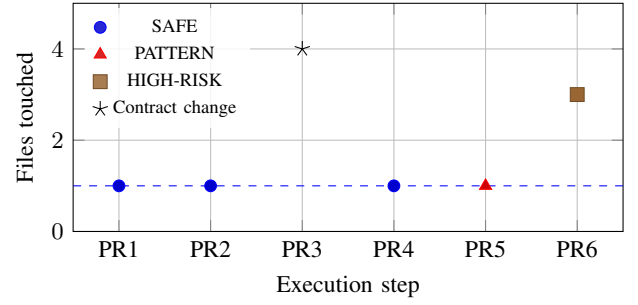


Fig. 3. RQ2 execution scope (blast radius): most steps are single-file; scope expands only when unavoidable (contract change in PR3) and for an isolated modular extraction (PR6).

breaking the software, but also observed that KatalIA consistently favored conservative changes and tended to defer pattern-level modifications. To explore the agent’s practical limits, we therefore explicitly prompted the application of the allowed design pattern (Template Method) after stable variation was confirmed, enabling a controlled assessment of pattern-guided recommendations under bounded scope. This manual prompt is reported as a controlled intervention and should be considered when interpreting pattern-level results.

d) Discussion: Overall, the results indicate that KatalIA’s recommendations are operationally actionable and context-aware: execution is dominated by conservative SAFE steps with a low blast radius, while escalation is explicit and controlled. In particular, broader changes were introduced only when the remediation required them (signature-level change in PR3) or when addressing higher-level modularization goals (PR6), reducing the risk of unbounded refactoring scope.

e) RQ3 Results: KatalIA’s application produced measurable post-refactoring changes in structural maintainability indicators and change-size proxies. We report pre-post deltas using the metrics snapshots collected on 2026-01-26 (PRE) and 2026-01-28 (POST). Because Git-history mining for the POST snapshot was unavailable (0 commits scanned for POST), change size is characterized using structural proxies (e.g., LOC redistribution, file-count growth, and modular extraction outcomes).

Table IV summarizes the outcomes for the primary refactored hotspot `app/adapters/repository/carro_repository.py`. The results show a strong maintainability improvement: MI increases from 31.92 to 59.11 (+85%), while duplication collapses (dup lines: 65→6; clone ratio: 0.2372→0.0571). Complexity also decreases (CC_{sum} : 44→25), and the file shrinks substantially (LOC: 561→137), consistent with a controlled extraction of responsibilities.

f) Discussion: Overall, the outcomes indicate that KatalIA can produce substantial maintainability gains under bounded scope when applied to a dominant hotspot: `carro_repository.py` shifts from a severe hotspot (MI=31.92) to a substantially healthier state (MI=59.11), with duplication reduced by more than 90% and complexity re-

Metric	PRE	POST	Δ	$\Delta\%$
Maintainability Index (MI)	31.92	59.11	+27.19	+85.2%
LOC	561	137	-424	-75.6%
SLOC	518	105	-413	-79.7%
CC_{sum}	44.0	25.0	-19.0	-43.2%
CC_{max}	5.0	4.0	-1.0	-20.0%
Clone ratio	0.2372	0.0571	-0.1801	-75.9%
Duplicated lines	65	6	-59	-90.8%
Duplicated blocks	29	2	-27	-93.1%
Magic number count	3	1	-2	-66.7%
Direct field access ratio	0.2971	0.0806	-0.2165	-72.9%
Hotspot score	415.82	94.46	-321.36	-77.3%

TABLE IV
RQ3 OUTCOMES FOR THE REFACTORED HOTSPOT
CARRO_REPOSITORY.PY: STRONG MI INCREASE WITH LARGE
REDUCTIONS IN DUPLICATION, COMPLEXITY, AND FILE SIZE.

duced by 43%. However, the post-refactoring MI remains below the threshold of 65, suggesting that further iterations are still needed to fully exit the hotspot set.

Regarding change size, the intervention exhibits a clear *modular redistribution* signature: the file shrinks dramatically (561→137 LOC) while new repository modules appear (`carro_repository_reads.py` and `carro_repository_writes.py`), and the analyzed file count increases (67→71). This indicates that improvements were achieved primarily by decomposing responsibilities rather than by “micro-tuning” within the same monolithic file. A key trade-off is that some duplication increases in `carro_repository_subsystems.py` (clone ratio 0.0973→0.177; dup lines 22→40), highlighting a next-step opportunity: consolidating shared subsystem routines without re-centralizing the logic.

Finally, the POST snapshot introduces auxiliary analysis/tooling modules (`smells/` and `tools/`) that appear as low-MI hotspots themselves. These are part of the refactoring infrastructure rather than the legacy service core and should be interpreted separately from production maintainability outcomes.

VI. CONCLUSIONS

This paper presented *Katalia*, a maintainability-aware refactoring recommender that integrates metric-based diagnosis, smell detection, hotspot ranking, and refactoring synthesis under explicit risk guardrails. The proposed pipeline aims to close a practical gap in legacy modernization by translating diagnostic evidence into conservative, incremental refactoring steps while preserving externally observable behavior.

The industrial case study provides structured evidence that *Katalia* can be operationally actionable in a realistic legacy setting. In particular, the intervention on the dominant hotspot (`carro_repository.py`) yielded substantial improvements in structural maintainability indicators, including a large increase in Maintainability Index and strong reductions in duplication and overall complexity, consistent with a controlled decomposition of responsibilities into more modular units. Practitioner feedback further suggests that the

recommendations were largely relevant and understandable, and that the SAFE-first ordering supported cautious execution under bounded scope.

Our findings complement and partially address gaps highlighted by the recent systematic literature review on LLM-based refactoring [5]. The SLR reports that many approaches remain weakly grounded in explicit maintainability objectives and often lack conservative change constraints, contributing to issues such as incorrect or incomplete transformations and limited robustness for complex refactorings. In contrast, *Katalia* operationalizes refactoring as an evidence-driven pipeline grounded on metric- and smell-based diagnosis and enforced by explicit risk guardrails (SAFE-first ordering, bounded scope, and build/test preservation), structuring execution into incremental PR-sized steps. At the same time, our results echo the SLR’s broader caution that improvements may not be uniform: while the dominant hotspot improved substantially, the post-refactoring snapshot suggests a redistribution of complexity/duplication into extracted modules and adjacent components, indicating the need for iterative follow-up.

Despite these positive outcomes, limitations apply. The evaluation is based on a single-case study, relies on structural proxies rather than causal outcome measures, and historical churn could not be computed in the available dataset, preventing history-based quantification of change-size risk. Finally, the SAFE-first strategy may defer pattern-level or more invasive architectural changes unless explicitly prompted.

Future work will strengthen external validity and expand outcome coverage by (i) replicating the evaluation across multiple industrial systems and technology stacks, (ii) enabling full Git-history mining to quantify bounded churn and history-based risk measures, (iii) comparing *Katalia* against baselines (manual refactoring and LLM-only recommendations) in controlled practitioner studies, and (iv) closing the feedback loop by learning from expert reviews to refine the knowledge base linking metrics, smells, refactorings, and patterns. Together, these extensions will help clarify when and how maintainability-aware, guardrail-driven LLM refactoring yields the highest practical benefit in high-risk legacy environments.

REFERENCES

- [1] H. M. Sneed and C. Verhoef, “Re-implementing a legacy system,” *Journal of Systems and Software*, vol. 155, pp. 162–184, 2019.
- [2] *ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*, International Organization for Standardization Std., 2011, accessed: 2025-07-18.
- [3] T. J. McCabe, “A complexity measure,” *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [4] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2004.
- [5] S. Martinez, L. Xu, M. Elnaggar, and E. Abdullah Alomar, “Software refactoring research with large language models: A systematic literature review,” *Journal of Systems and Software*, vol. 235, p. 112762, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121225004315>
- [6] M. O’Keeffe and M. Cinnéide, “Search-based refactoring for software maintenance,” *Journal of Systems and Software*, vol. 81, no. 4, pp. 502–516, 2008.

- [7] A. Martini, J. Bosch, and M. Chaudron, "Investigating architectural technical debt accumulation and refactoring over time: A multiple-case study," *Information and Software Technology*, vol. 67, pp. 237–253, 2015.
- [8] B. Liu, Y. Jiang, Y. Zhang, N. Niu, G. Li, and H. Liu, "Exploring the potential of general purpose LLMs in automated software refactoring: an empirical study," *Automated Software Engineering*, vol. 32, no. 1, p. 26, May 2025.
- [9] L. Yu, E. Alégroth, P. Chatzipetrou, and T. Gorschek, "Evaluating the quality of genai applications in software engineering: A multi-case study," *Empirical Software Engineering*, vol. 31, p. 29, 2026.
- [10] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012.
- [11] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, 2003.
- [12] I. Yanakiev, B. M. Lazar, and A. Capiluppi, "Applying solid principles for the refactoring of legacy code: An experience report," *Journal of Systems and Software*, vol. 220, p. 112254, 2 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S016412122400298X>
- [13] "Refactoring of legacy software using model learning and equivalence checking: An industrial experience report," in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 9681. Springer Verlag, 2016, pp. 311–325.
- [14] N. Nagappan and T. Ball, "Use of relative code churn measures to predict system defect density," in *Proceedings of the 27th International Conference on Software Engineering*. ACM, 2005, pp. 284–292.
- [15] F. Khomh, M. Di Penta, Y.-G. Guéhéneuc, and G. Antoniol, "An exploratory study of the impact of code smells on software change-proneness," in *2009 16th Working Conference on Reverse Engineering*. IEEE, 2009, pp. 75–84.
- [16] M. A. S. Bigonha, K. Ferreira, P. Souza, B. Sousa, M. Januário, and D. Lima, "The usefulness of software metric thresholds for detection of bad smells and fault prediction," *Information and Software Technology*, vol. 115, pp. 68–81, 2019.
- [17] T.-H. Chen, C.-L. Tsai, S.-Y. Hsieh, and Y. Kamei, "Understanding metric-based detectable smells in Python software: A comparative study," *Information and Software Technology*, vol. 94, pp. 14–29, 2018.
- [18] A. Yamashita and S. Counsell, "Code smells and their relationship with maintainability: A large-scale empirical study," *Journal of Systems and Software*, vol. 86, no. 10, pp. 2639–2653, 2013.
- [19] B. Walter and T. Alkhaier, "The relationship between design patterns and code smells: An exploratory study," *Information and Software Technology*, vol. 74, pp. 127–142, 2016.
- [20] F. Batole, D. OBrien, T. N. Nguyen, R. Dyer, and H. Rajan, "An LLM-based agent-oriented approach for automated code design issue localization," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE/ACM, 2025.
- [21] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, p. 220, December 2024.
- [22] Z. Li *et al.*, "A transformer-based approach for code smell detection," *Journal of Systems and Software*, vol. 190, 2022.
- [23] D. A. Gatica Pizarro, "Katalia artifacts (metrics and knowledge base)," Zenodo, 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.18408185>
- [24] M. Heričko, A. Živković, and B. Robič, "On using the maintainability index for assessing software technical debt: An empirical study," *Empirical Software Engineering*, vol. 28, no. 1, p. 15, 2023.
- [25] D. A. Gatica Pizarro, "Katalia: Tool, prompts, and replication package," GitHub repository, 2026, accessed: 2026-02-01. [Online]. Available: <https://github.com/MrNeo44/katalia>
- [26] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [27] Y. Kamei, S. Matsumoto, A. Monden, K.-i. Matsumoto, B. Adams, and A. E. Hassan, "Revisiting common bug prediction findings using effort-aware models," in *Proceedings of the 2010 IEEE International Conference on Software Maintenance (ICSM)*. IEEE, 2010, pp. 1–10.